

Exercise 7: Financial Forecasting

Scenario:

You are developing a financial forecasting tool that predicts future values based on past data.

Steps:

1. Understand Recursive Algorithms:

- Explain the concept of recursion and how it can simplify certain problems.

2. Setup:

- Create a method to calculate the future value using a recursive approach.

3. Implementation:

- Implement a recursive algorithm to predict future values based on past growth rates.

4. Analysis:

- Discuss the time complexity of your recursive algorithm.
- Explain how to optimize the recursive solution to avoid excessive computation.

Understanding Recursive Algorithms

Recursion is a programming technique where a method calls itself to solve a smaller instance of the same problem. It simplifies complex problems by breaking them down into identical, manageable sub-problems. Every recursive function requires a Base Case (a condition to stop the recursion) and a Recursive Case (the logic that moves the problem closer to the base case).

CODE:

FinancialForecasting.java

```
J FinancialForecasting.java
1 public class FinancialForecasting {
2
3     public static double calculateFutureValue(double presentValue, double growthRate, int periods) {
4         if (periods == 0) {
5             return presentValue;
6         }
7         return calculateFutureValue(presentValue * (1 + growthRate), growthRate, periods - 1);
8     }
9
10    public static void main(String[] args) {
11        double presentValue = 1000.0;
12        double pastDataGrowthRate = 0.05;
13        int periodsToForecast = 10;
14
15        System.out.println("--- Financial Forecasting ---");
16        System.out.println("Present Value: $" + presentValue);
17        System.out.println("Growth Rate: " + (pastDataGrowthRate * 100) + "%");
18        System.out.println("Periods to Forecast: " + periodsToForecast + " years");
19
20        double futureValue = calculateFutureValue(presentValue, pastDataGrowthRate, periodsToForecast);
21
22        System.out.printf("\nPredicted Future Value: $%.2f\n", futureValue);
23    }
24 }
```

OUTPUT:

```
--- Financial Forecasting ---
Present Value: $1000.0
Growth Rate: 5.0%
Periods to Forecast: 10 years

Predicted Future Value: $1628.89
```

Analysis:**Time Complexity**

The time complexity of this specific recursive algorithm is $O(n)$, where 'n' is the number of periods. This is because the function makes exactly one recursive call for each period until it hits the base case.

Optimization

In simple linear recursion like this, the main risk is a `StackOverflowError` if the number of periods is extremely high. However, in more complex financial modeling where recursive calls branch out (e.g., exploring multiple investment scenarios resulting in an $O(2^n)$ complexity), we optimize using Memoization. Memoization stores the results of expensive function calls in a data structure (like a `HashMap`) and returns the cached result when the same inputs occur again, preventing redundant calculations and drastically improving performance.